AMATH 482: HOME WORK 3

JIAYUAN LI

Amath Department, University of Washington, Seattle, WA
jli72@uw.edu

Abstract. In this project, we implemented and compared several regression models of increasing complexity to predict wine quality from standardized chemical features. Ordinary Least Squares (OLS), quadratic expansions, and interaction models were evaluated using Mean Squared Error (MSE) on a held-out test set. To control model complexity in the expanded feature space, Lasso regularization with cross-validation was applied. Model selection behavior was visualized, and prediction performance was analyzed using residual diagnostics. All computations and visualizations were performed in Python using NumPy, Scikit-learn, and Matplotlib.

1. Introduction and Overview

Linear regression is one of the fundamental tools in data-driven modeling and statistical learning. Given a set of input features and a corresponding response variable, the objective is to determine a linear mapping that best approximates the relationship between them. This problem is commonly formulated as a least-squares optimization problem, where the squared prediction error is minimized. A general discussion of regression methods within data-driven modeling can be found in [3].

In this homework, we implemented several regression models of increasing complexity. We first constructed a standard first-order linear model using Ordinary Least Squares (OLS). To capture potential nonlinear effects, the feature space was expanded to include quadratic terms and selected interaction terms. To control model complexity and reduce overfitting in the expanded feature space, Lasso regularization with cross-validation was applied to select the regularization parameter.

All computations were carried out in Python using NumPy and Scikit-learn [1, 4]. Model performance was evaluated using Mean Squared Error (MSE) on a held-out test set, and the models were compared based on their generalization performance.

2. Theoretical Background

2.1. **Standardization.** Let $X \in \mathbb{R}^{n\times p}$ denote the feature matrix and $y \in \mathbb{R}^n$ the response vector. To improve numerical stability and ensure comparability across features, each feature column was standardized using training-set statistics:

$$x^{(std)} = \frac{x-\mu}{\sigma},$$

where μ and σ denote the training mean and standard deviation. The same transformation was applied to the response variable. This ensures that all variables have zero mean and unit variance during training.

Date: February 25, 2026.

2.2. **Ordinary Least Squares.** The Ordinary Least Squares (OLS) estimator minimizes the squared residual norm:

$$\min_{\beta} \|X\beta - y\|_2^2.$$

Setting the gradient to zero yields the normal equation:

$$X^T X\beta = X^T y.$$

If $X^T X$ is invertible, the solution is

$$\beta = (X^T X)^{-1} X^T y.$$

This formulation was used for both the first-order model and the expanded feature models.

2.3. **Feature Expansion.** To increase model flexibility, the feature space was expanded in two ways:

Quadratic terms (no interactions). Each feature was squared to construct a second-order model while keeping the regression linear in parameters.

Interaction terms. Selected pairwise products $x_i x_j$ were added to capture dependencies between features.

Although these models are nonlinear in the original variables, they remain linear in the coefficient vector and can therefore be solved using OLS.

2.4. **Lasso Regularization.** When the feature space becomes large, overfitting may occur. Lasso regression adds an ℓ_1 penalty to the least-squares objective:

$$\min_{\beta} \frac{1}{n}\|X\beta - y\|_2^2 + \alpha\|\beta\|_1.$$

The parameter α controls the strength of regularization. Cross-validation was used to select the optimal α, after which the final model was evaluated on the test set.

2.5. **Model Evaluation.** All models were compared using Mean Squared Error (MSE):

$$\text{MSE} = \frac{1}{n}\|y - \hat{y}\|_2^2.$$

The model with the lowest test MSE was selected as the best-performing model.

3. Algorithm Implementation and Development

All models were implemented in Python using matrix-based computations. Data preprocessing, including standardization and train-test splitting, was performed prior to model fitting. Feature expansions (quadratic terms and selected interaction terms) were constructed explicitly by augmenting the design matrix.

For the OLS models, the normal equation was implemented directly using NumPy linear algebra routines [1]. Matrix multiplications and linear system operations were computed using efficient array operations.

For the regularized model, Lasso regression with cross-validation was implemented using the LassoCV functionality provided by Scikit-learn [4]. Cross-validation was used to automatically select the optimal regularization parameter α, after which the model was refit on the full training set.

Figures were generated using Matplotlib [2].

4. Computational Results

Standardization Rule. The training features and outputs were standardized to have zero mean and unit variance. The test set was transformed using the mean and standard deviation computed from the training set only. No scaling parameters were fitted on the test data.

First-order OLS. The first-order linear model was fit using ordinary least squares. The resulting test MSE is

$$\mathrm{MSE}_{\mathrm{first}} = 0.747169690519.$$

The five predictors with the largest coefficient magnitudes are

$$[10, 1, 9, 6, 4].$$

Second-order Model (No Interactions). A quadratic model including squared terms (but no interaction terms) was fit using OLS. The resulting test MSE is

$$\mathrm{MSE}_{\mathrm{quad}} = 0.780582119213.$$

Since this error is larger than that of the first-order model, including quadratic terms alone does not improve generalization performance for this dataset.

Interaction Model (OLS). Using the index set

$$S = [1, 4, 6, 9, 10],$$

we constructed a model including all first-order terms and all pairwise interaction terms among indices in S. The resulting test MSE is

$$\mathrm{MSE}_{\mathrm{int}} = 0.717522472822.$$

Including interaction terms significantly reduces the test error compared to both the first-order and quadratic models.

Interaction Model with Lasso Regularization. Lasso regression with 5-fold cross-validation was applied to the interaction model. Figure 1 shows the cross-validated MSE as a function of the regularization parameter α (on a log scale). The selected value is

$$\alpha = 0.00969665789314552,$$

which lies near the minimum of the cross-validation curve and reflects a balance between model complexity and regularization strength.

Refitting the Lasso model using the selected α yields a test MSE of

$$\mathrm{MSE}_{\mathrm{lasso}} = 0.704284839101,$$

which is the lowest among all models considered.

The nonzero coefficients in the refitted Lasso model are listed in Table 1.

Prediction Analysis. Figure 2 shows the test-set predictions versus true values for the interaction Lasso model. Most points cluster around the diagonal line $y_{\mathrm{pred}} = y_{\mathrm{true}}$, indicating good predictive alignment. The residual mean is close to zero (0.024), suggesting no systematic bias, while the residual standard deviation (0.670) reflects the spread contributing to the test MSE.

Model Comparison. A summary of test MSE values is given in Table 2.

The interaction Lasso model achieves the lowest test MSE among all models considered. These results indicate that interaction effects are beneficial for this dataset and that ℓ_1 regularization improves generalization by controlling model variance.

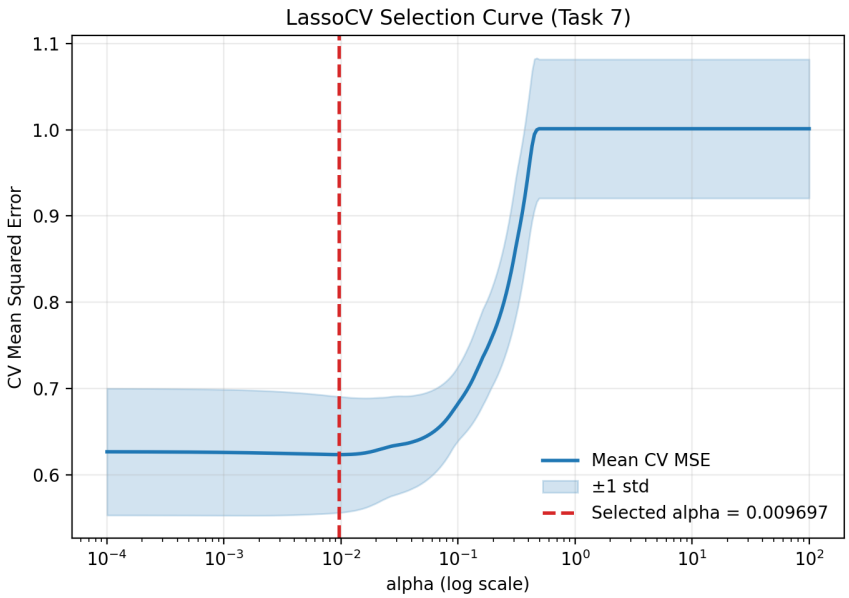

FIGURE 1. Cross-validated MSE versus regularization parameter α for the interaction Lasso model. The dashed line indicates the selected α from 5-fold cross-validation. Shaded region represents ± 1 standard deviation across folds.

Feature	Coefficient
x1	-0.2553
x2	-0.0694
x4	-0.0739
x5	0.0607
x6	-0.1755
x8	-0.0682
x9	0.2738
x10	0.3242
x1*x6	0.0875
x1*x9	0.0041
x4*x9	-0.0242
x4*x10	-0.0045
x6*x9	-0.0756
x9*x10	0.0365

TABLE 1. Nonzero coefficients in the refitted interaction Lasso model.

Model	Test MSE
First-order OLS	0.747169690519
Quadratic OLS	0.780582119213
Interaction OLS	0.717522472822
Interaction Lasso	0.704284839101

TABLE 2. Comparison of test MSE across models (lower is better).

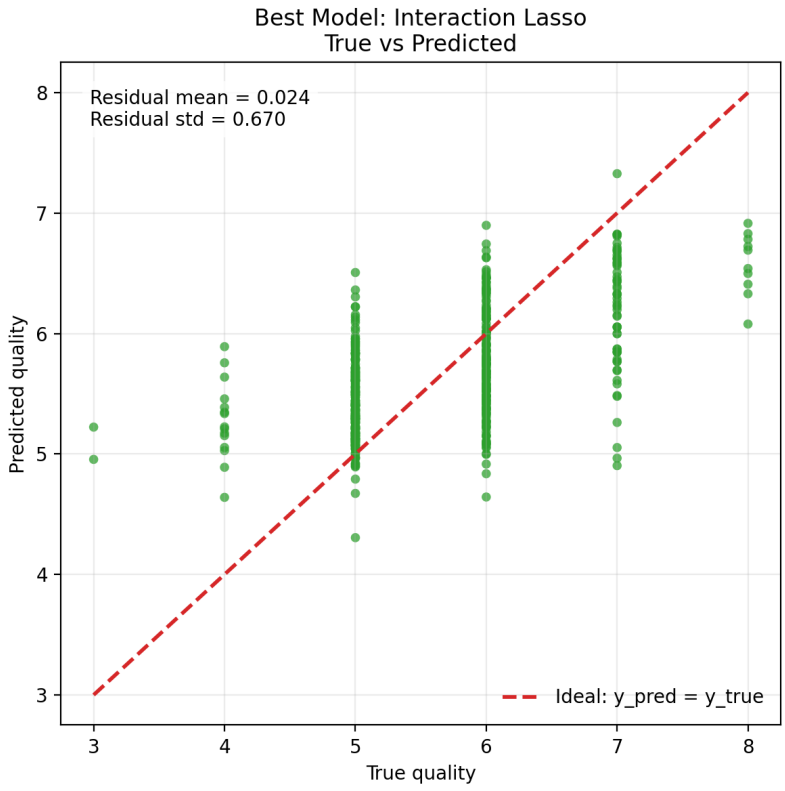

FIGURE 2. Standardized predicted versus true wine quality on the test set for the interaction Lasso model. The dashed line represents perfect prediction.

5. Summary and Conclusions

In this homework, we constructed and compared a sequence of regression models of increasing complexity to predict wine quality from chemical measurements. After standardizing the data using training-set statistics, we first fit a first-order linear model, followed by a quadratic model without interactions, an interaction OLS model, and finally an interaction model with Lasso regularization.

The quadratic model did not improve performance relative to the first-order model, indicating that adding squared terms alone does not capture additional predictive structure in this dataset. However, incorporating interaction terms significantly reduced the test error, suggesting that dependencies between certain features play an important role in predicting wine quality.

Among all models considered, the interaction Lasso model achieved the lowest test MSE. This demonstrates that combining interaction terms with ℓ_1 regularization provides a good balance between model flexibility and generalization. Overall, the results indicate that including carefully selected interaction effects, together with regularization, leads to improved predictive performance for this problem.

Acknowledgments

The author would like to thank Professor Natalie Frank for her lectures on regression methods and regularization, which provided the theoretical foundation for this work. Special thanks to the Teaching Assistants for helpful clarifications on model selection and implementation details during office hours and on Piazza. The author also appreciates discussions with classmates regarding cross-validation strategies and feature expansion.

References

[1] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, et al. Array programming with numpy. *Nature*, 585(7825):357–362, 2020.
[2] J. D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.
[3] J. N. Kutz. *Data-driven modeling & scientific computation: methods for complex systems & big data*. OUP Oxford, 2013.
[4] F. Pedregosa, G. Varoquaux, A. Gramfort, et al. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.